```
cat /dev/LFY_device
```

It will give us the output shown in Figure 6.

One point we missed out in our discussion is the *copy_from_user* (*asm/uaccess.h*). And here is the code for it:

```
static inline long copy_from_user(void *to,
        const void __user * from, unsigned long n)
{
      might_sleep();
      if (access_ok(VERIFY_READ, from, n))
        return __copy_from_user(to, from, n);
      else
        return n;
}

static inline long copy_to_user(void __user *to,
        const void *from, unsigned long n)
{
      might_sleep();
      if (access_ok(VERIFY_WRITE, to, n))
        return __copy_to_user(to, from, n);
      else
        return n;
}
```

You can use the code given above to comprehend how the actual process works.

One new header file that we added in this tutorial is the *errno.h* file. And here are the 'important numbers' from the file:

```
#define ERESTARTSYS512
#define ERESTARTNOINTR  513
#define ERESTARTNOHAND  514 /* restart if no handler.. */
#define ENOIOCTLCMD515  /* No ioctl command */
#define ERESTART_RESTARTBLOCK 516 /* restart by calling sys_restart_syscall */
#define EBADHANDLE 521  /* Illegal NFS file handle */
#define ENOTSYNC    522  /* Update synchronization mismatch */
#define EBADCOOKIE 523  /* Cookie is stale */
#define ENOTSUPP    524  /* Operation is not supported */
#define ETOOSMALL   525  /* Buffer or request is too small */
#define ESERVERFAULT    526  /* An untranslatable error occurred */
#define EBADTYPE    527  /* Type not supported by server */
#define EJUKEBOX    528  /* Request initiated, but will not complete before
timeout */
#define EIOCBQUEUED529  /* iocb queued, will get completion event */
#define EIOCBRETRY 530  /* iocb queued, will trigger a retry */
```

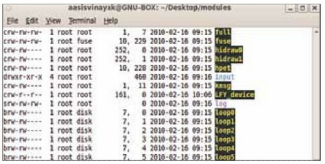

Figure 5: Listing the new file created

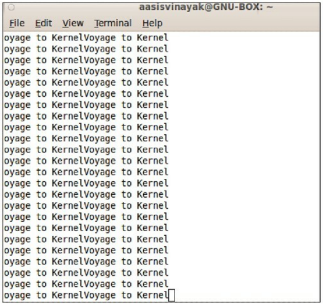

Figure 6: Output of cat /dev/LFY_device

Now we know how to write a driver for a device. If you wish to dig deeper into the subject by making a simple piece of actual hardware and controlling it with a Linux driver, please let me know. I will include such experiments in the upcoming days of the voyage.

Happy hacking! END

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com

A **Computer Magazine** that does not talk about Music, Games, Photo albums—**Only Business**

Read BenefIT: A Business Minded Computer Magazine

www.benefitmag.com